

StockFlow Case Study

Name : Abhishek Dani

This is my solution to the StockFlow backend case study.

I have tried to keep the implementation simple while clearly explaining the reasoning behind each decision.

Since the requirements were intentionally incomplete, I made a few assumptions upfront and focused on building a clean, correct, and explainable system.

Assumptions I Made

- SKUs are globally unique across the platform
- A product can exist without being assigned to a warehouse initially
- Low-stock alerts are generated per warehouse
- “Recent sales” means at least one sale in the last 30 days
- A product can have multiple suppliers, but one is returned
- If there are no recent sales, no alert should be generated

Part 1 : Code Review

Here are the key issues I found in the original API:

1. Two Separate Commits

- The product and inventory were committed separately.
- If inventory creation fails, the product still exists.
- This leads to inconsistent data.

Fix: Wrap both operations in a single transaction using `flush()` and commit once.

2. Product Tied to Single Warehouse

- The original design assumed one warehouse per product.
- This conflicts with requirement: products can exist in multiple warehouses

Fix: Use a separate Inventory table to map product ↔ warehouse.

3. No Input Validation

- The API directly accessed fields like `data['name']`.
- Missing fields would crash the API.

Fix: Validate required fields (name, sku, price) before processing.

4. SKU Uniqueness Not Enforced

- Duplicate SKUs could be inserted.
- Causes reporting and billing issues.

Fix: Check for existing SKU before insert.

5. Price Stored as Float

- Float causes precision errors.
- Example: financial calculations become inaccurate.

Fix: Use Decimal for price.

6. No Warehouse Validation

- Invalid warehouse IDs could be used.
- Leads to broken relationships.

Fix: Validate warehouse before creating inventory.

7. No Error Handling

- Failures would crash the API.
- Poor user experience.

Fix: Use try/except with rollback.

8. No Idempotency

- Repeated requests could create duplicate data.
- Leads to inconsistent system behavior.

Summary

The main focus of fixes was:

- Data consistency
- Validation
- Correct data modeling
- Error handling

Part 2: Database Design

The screenshot displays the MySQL Workbench interface with a local instance of MySQL80. The left sidebar shows the 'SCHEMAS' pane with a tree view of databases including 'abhi', 'abhibd', 'bynry', 'carinsurancedb', 'countries', 'countriesess', 'librarydb', 'librarydbb', 'pets', 'sys', and 'university'. The 'bynry' database is selected, and the 'Tables' folder is expanded. The main editor window shows the SQL script for creating the database schema.

```
1
2
3 • CREATE DATABASE IF NOT EXISTS bynry;
4 • USE bynry;
5
6
7
8 • CREATE TABLE IF NOT EXISTS companies (
9   id CHAR(36) PRIMARY KEY,
10  name VARCHAR(255) NOT NULL,
11  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
12 );
13
14 • CREATE TABLE IF NOT EXISTS warehouses (
15   id CHAR(36) PRIMARY KEY,
16   company_id CHAR(36),
17   name VARCHAR(255) NOT NULL,
18   location TEXT,
19   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
20   FOREIGN KEY (company_id) REFERENCES companies(id)
21   ON DELETE CASCADE
22 );
23
24
25
26 • CREATE TABLE IF NOT EXISTS products (
27   id CHAR(36) PRIMARY KEY,
28   company_id CHAR(36),
29   name VARCHAR(255) NOT NULL,
30   sku VARCHAR(100) NOT NULL UNIQUE,
31   price DECIMAL(10,2) NOT NULL,
32   low_stock_threshold INT DEFAULT 10,
33   is_bundle BOOLEAN DEFAULT FALSE,
34   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
35   FOREIGN KEY (company_id) REFERENCES companies(id)
36   ON DELETE CASCADE
37 );
38
39
40 • CREATE TABLE IF NOT EXISTS inventory (
41   id CHAR(36) PRIMARY KEY,
42   product_id CHAR(36),
43   warehouse_id CHAR(36),
44   quantity INT DEFAULT 0,
45   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
46
47   UNIQUE(product_id, warehouse_id),
48
49   FOREIGN KEY (product_id) REFERENCES products(id)
50   ON DELETE CASCADE,
51   FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
52   ON DELETE CASCADE
53 );
54
55
56 • CREATE TABLE IF NOT EXISTS inventory_logs (
57   id CHAR(36) PRIMARY KEY,
58   product_id CHAR(36),
59   warehouse_id CHAR(36),
60   change_quantity INT,
61   reason VARCHAR(255),
62   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
63
64   FOREIGN KEY (product_id) REFERENCES products(id),
65   FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
66 );
67
68
69 • CREATE TABLE IF NOT EXISTS suppliers (
70   id CHAR(36) PRIMARY KEY,
71   name VARCHAR(255) NOT NULL,
72   contact_email VARCHAR(255),
73   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
74 );
75
```

```
76
77 • CREATE TABLE IF NOT EXISTS product_suppliers (
78     product_id CHAR(36),
79     supplier_id CHAR(36),
80
81     PRIMARY KEY (product_id, supplier_id),
82
83     FOREIGN KEY (product_id) REFERENCES products(id)
84     ON DELETE CASCADE,
85     FOREIGN KEY (supplier_id) REFERENCES suppliers(id)
86     ON DELETE CASCADE
87 );
88
89
90 • CREATE TABLE IF NOT EXISTS product_bundles (
91     parent_product_id CHAR(36),
92     child_product_id CHAR(36),
93     quantity INT DEFAULT 1,
94
95     PRIMARY KEY (parent_product_id, child_product_id),
96
97     FOREIGN KEY (parent_product_id) REFERENCES products(id),
98     FOREIGN KEY (child_product_id) REFERENCES products(id)
99 );
100
101
102 • CREATE TABLE IF NOT EXISTS sales (
103     id CHAR(36) PRIMARY KEY,
104     product_id CHAR(36),
105     quantity INT,
106     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
107
108     FOREIGN KEY (product_id) REFERENCES products(id)
109 );
110
111
112 • SHOW TABLES;
113 • DESCRIBE products;
```

```

114 • SHOW CREATE TABLE inventory;
115
116
117
118 • INSERT IGNORE INTO companies (id, name)
119 VALUES ('c1', 'Demo Company');
120
121 • INSERT IGNORE INTO warehouses (id, company_id, name)
122 VALUES ('w1', 'c1', 'Main Warehouse');
123
124 • INSERT IGNORE INTO products (id, company_id, name, sku, price, low_stock_threshold)
125 VALUES ('p1', 'c1', 'Widget A', 'WID-001', 100.00, 20);
126
127 • INSERT IGNORE INTO inventory (id, product_id, warehouse_id, quantity)
128 VALUES ('i1', 'p1', 'w1', 5);
129
130 • INSERT IGNORE INTO suppliers (id, name, contact_email)
131 VALUES ('s1', 'Supplier Corp', 'orders@supplier.com');
132
133 • INSERT IGNORE INTO product_suppliers (product_id, supplier_id)
134 VALUES ('p1', 's1');
135
136 • INSERT IGNORE INTO sales (id, product_id, quantity, created_at)
137 VALUES ('sale1', 'p1', 10, NOW());
138
139
140 • SELECT
141     p.name,
142     i.quantity,
143     p.low_stock_threshold
144 FROM products p
145 JOIN inventory i ON p.id = i.product_id
146 WHERE i.quantity < p.low_stock_threshold;

```

Questions I would ask the product team:

1. Is low-stock threshold defined per product or per warehouse ?
2. What is the exact definition of “recent sales” (fixed 30 days or configurable) ?
3. Can a product have multiple suppliers, and how do we choose one (cheapest, preferred, fastest) ?
4. Can inventory go negative (for backorders) or must it always stay ≥ 0 ?
5. Should returns, transfers, or damages also update inventory or only sales ?
6. Is authentication required to restrict access to company-specific data ?
7. How should bundle products behave (reduce component stock automatically or not) ?

Design decisions and justification :

- Used a separate Inventory table to handle product–warehouse mapping because a product can exist in multiple warehouses.
- Used an InventoryMovement table to track stock changes over time, which helps in calculating sales rate and analyzing trends.
- Enforced unique SKU to prevent duplicate products and maintain consistency.
- Stored price as Decimal instead of float to avoid precision issues in financial calculations.
- Used a SupplierProduct table to support multiple suppliers per product and flexible sourcing.
- Added indexing on product_id, warehouse_id, and created_at to make stock lookup and recent sales queries faster.
- Followed normalization to avoid redundancy and keep the schema scalable and maintainable.

Part 3: Low-Stock Alerts API

Endpoint:

GET /api/companies/{company_id}/alerts/low-stock

The API returns products that are low on stock and require attention.

Logic used:

- For each product in the company, check its inventory across all warehouses
- Only consider entries where stock is less than or equal to the product's threshold
- From those, include only products that have at least one sale in the last 30 days
- Calculate average daily sales using total sales in last 30 days
- Estimate days until stockout using:
 $\text{current_stock} / \text{average_daily_sales}$
- Fetch supplier information for that product (if available)

Response includes:

- product_id, product_name, sku
- warehouse_id
- current_stock and threshold
- days_until_stockout
- supplier details (if exists, otherwise null)

Edge cases handled:

- If no recent sales → product is skipped (no alert)
- If average daily sales = 0 → days_until_stockout is set to null (avoid division error)
- If no supplier found → supplier field is null
- If product exists in multiple warehouses → each warehouse is evaluated separately
- If company has no products → empty alerts list returned

Reasoning:

- Only showing low stock is not enough; filtering by recent sales ensures alerts are actionable
- Separating inventory and movement data helps calculate sales trends efficiently
- Calculating days until stockout helps prioritize urgent items
- Keeping logic simple ensures readability and maintainability while still solving the business problem